

uptodate: a friendly and smart job board for freshers looking for tech jobs offcampus

Live: <https://uptodate.dfaqq1w4pxpfm.amplifyapp.com> **Repo:** <https://github.com/Naman-GG/uptodate> **Track:** Ship It · solo

Longer technical detail: [DETAILS.md](#) · non-technical walkthrough: [HOW-IT-WORKS.md](#)

The problem

I am a final-year engineering student looking for an AI/ML job or internship in India. the search is not hard because there is nothing out there. it is hard because almost everything out there is not for me and no filter will tell me which.

Search “AI Intern” on any board and you get a wall of results. most are US roles that say “eligible for US students” somewhere deep in the JD, or need 1+ YOE, or are not engineering roles at all, just something unrelated masked under a tech title. a meaningful fraction are training institutes selling certificates, posting the same advert under a dozen IDs.

The information that would let you skip all of it **is in the description**, phrased differently every time. job boards filter on titles and tags so they cannot see it. you find out by opening two hundred tabs.

What I built

Four stages, each its own Lambda, orchestrated by Step Functions on a daily EventBridge schedule.

Ingest pulls structured JSON from Greenhouse, Lever and Ashby board APIs plus Adzuna for India coverage. 201 boards, no HTML scraping. Cross-source dedup collapses the same role appearing twice.

Prescreen drops titles no fresher could take (Senior, Staff, Engineer III, clearly non-technical families) using plain string matching. 14,547 down to 3,713 before a single token is spent.

Extract sends each remaining description to Bedrock and gets back structured facts: experience required, countries, work-authorisation constraint, role family, whether it is actually technical. Structure is guaranteed by forcing a tool call whose input schema is the target shape. Our own validator re-checks every field and anything that fails is quarantined. Real failure rate: 13 of 3,700.

Gate applies your criteria to those facts in plain code and records why each rejection happened. **The gate function holds no bedrock:*** IAM permission, so it structurally cannot ask a model whether you qualify. That is why “requires 3 years experience” is a fact from the posting rather than a model’s opinion.

Score runs a Strands agent on survivors only, writing a fit score and a one-line justification.

Your profile drives all of it and is editable in the UI. upload a resume PDF and Bedrock reads it directly to fill the fields in.

What it actually did

Stage	Count
Scanned	14,547
Screened out in code	10,834
Read by Bedrock	3,700
Eligible	154

A 98.9% reduction, every step explainable. the rejection reasons are the argument: **1,296** were not technical roles, **840** required US work authorisation, 1,326 wanted three years. all three facts live only in the prose.

Where AWS fits

The whole stack is one SAM template. nine services, each doing a specific job.

Amazon Bedrock is the reasoning layer, with two models for two jobs. *Nova 2 Lite* reads all 3,700 job descriptions, it is the cheap model and it runs on every posting so its cost sets the bill. *Nova Pro* handles the work that runs rarely and needs judgement: scoring the 154 survivors and reading uploaded resumes. both go through the **Converse API** rather than a provider-specific endpoint, which is what lets the model be a config value. structured output comes from forcing a tool call whose schema is the shape we want, and resumes are sent as Converse document blocks so the PDF goes to Bedrock without any parsing library.

AWS Lambda runs five functions, kept separate so a failure or a prompt change only replays the stage that needs it.

Function	Role
<code>ingest</code>	fetches 201 boards plus Adzuna, dedups, writes to DynamoDB
<code>extract</code>	sends each description to Bedrock, validates, quarantines failures
<code>gate</code>	applies the eligibility rules in code and records every reason
<code>score</code>	runs the Strands agent on survivors only
<code>api</code>	serves the board, funnel, profile and resume endpoints

Amazon DynamoDB holds three tables: `raw_postings` (write-once source payloads with a 60-day TTL, so a prompt change can be replayed against the original text), `postings` (the enriched record), and `profiles`. two indexes do the real work: `by-profile-fit` serves the board already ordered by score, and `by-stage` lets each pipeline stage find its backlog without a table scan.

AWS IAM is where the architecture is actually enforced. each function has its own role, and the gate function's role contains **no bedrock:* permission at all**. it is not that the gate chooses not to call a model, it cannot. that is a property of the deployed configuration rather than a promise about the code.

AWS Step Functions chains ingest → extract → gate → score with per-stage retry policies, looping back while `has_more` is true so a large backlog drains across invocations.

Amazon EventBridge fires the pipeline daily at 07:00 IST. it ran overnight without being asked, which is how I confirmed it was wired correctly.

Amazon API Gateway fronts the read/write Lambda: `GET /postings`, `PATCH /postings/{id}`, `GET /funnel`, `GET/PUT /profile`, `POST /profile/suggest`.

AWS Amplify hosts the React frontend, deployed from the CLI rather than a connected Git repo.

CloudWatch Logs is where the IAM failure surfaced with the exact denied action, and **AWS Budgets** holds a \$25 alarm against a runaway debugging loop.

Cost: \$0.00. extraction runs about \$5.50 on Nova 2 Lite and credits covered it. the prefilter is why that number is small: dropping 10,834 postings with string matching before any model sees them is the difference between roughly \$22 and \$5.50.

What I learned

Third-party models on Bedrock are Marketplace purchases with their own payment rules.

Anthropic models need a Marketplace subscription requiring a credit card. mine is a debit card and Marketplace in India does not accept UPI, so it failed with `INVALID_PAYMENT_INSTRUMENT` in a loop AWS threads describe as needing support escalation. I rewrote the model layer onto the **Converse API**, the cross-model interface, and moved to Amazon Nova which AWS serves directly. the model became an environment variable instead of an architectural commitment, and extraction got *cheaper*: \$5.50 against roughly \$16. the constraint produced a better design than the one I set out with.

A Default: in a CloudFormation template only applies at stack creation. after that the stack reuses the value it was given. I changed the default model, redeployed without passing it, and the Lambda ran against a model the account cannot reach, quarantining 600 postings in ten seconds. `sam build` and `sam deploy` both reported success because neither runs the code.

Least privilege caught a real bug. the scorer's IAM role allowed the postings table but not the raw table where descriptions live, so it crashed. had I given every function blanket DynamoDB access it would have worked silently, and so would the gate, whose whole guarantee is that it cannot reach Bedrock.

A DynamoDB query returns at most 1 MB and `Select=COUNT` counts only that page. the funnel reported 2,553 when the table held 14,526. the number looked plausible, which is what made it dangerous.

What is not finished

I tried to detect fake employers and shipped nothing. seven attempts, five prompt revisions and two deterministic rules, every one confidently wrong in a different direction: calling real companies

fake for unfamiliar names, then flagging 89% of the board. the code rules were worse, flagging Databricks, OpenAI and Stripe because large employers genuinely repeat titles across locations. the real pattern needs descriptions compared across the whole corpus, a different shape of problem. so I removed it. a flag that mislabels Stripe teaches you to distrust every other signal on the card.

The reasoning lines are decent, not great. four rounds got them naming what the posting asks for, but the second half still restates the profile rather than assessing the gap.

Single profile. the pipeline is parameterised by `profile_id` and the profile is stored server-side, so multi-user is a Cognito integration rather than a rewrite. researched it, ran out of time.